GSoC 2026 Project Proposal: Improving MP-BioPath via Automated Genomic Integration and Quantitative Perturbation Analysis

Organization: Open Genome Informatics (Stein Lab / Reactome)
Project Length: 175 Hours (Medium Scope)
Candidate: Julian Benson
Background: B.S. Biochemistry & Molecular Biology, Commonwealth Honors College, UMass Amherst

---

1. Executive Summary & Project Abstract

The MP-BioPath algorithm has successfully demonstrated that manually curated Reactome logic graphs can predict the downstream effects of biological perturbations with ~75% accuracy across thousands of test cases (Wright et al., 2022). However, the tool currently operates primarily as a retrospective literature-validation engine, requiring manual definition of root inputs (RI) and key outputs (KO).

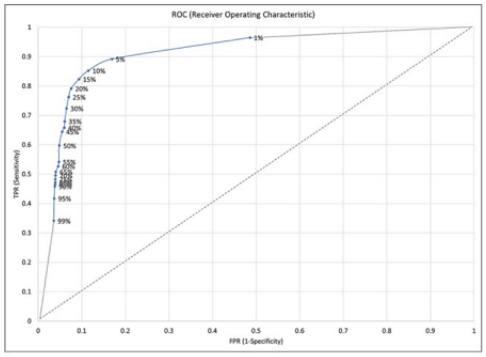
To transition MP-BioPath into a high-throughput predictive engine for modern bioinformatics, this project will develop a polyglot (Python/R/Julia) integration pipeline. By automating the ingestion of quantitative genomic data (e.g., RNA-seq differential expression), mathematically mapping it to Reactome's continuous variable bounds, and resolving known inaccuracies regarding "Entity Set Dilution," Gene Expression Regulation (GER), and Ubiquitous Substrate computation, this project will enable researchers to rapidly predict pathway-level consequences of patient-specific genomic profiles.

---

2. Background & Biological Bottlenecks

2.1 The MP-BioPath Baseline & Optimization Thresholds

MP-BioPath translates Reactome pathways into Logical Networks. In its 2022 evaluation, the Stein Lab evaluated 18,539 test cases against experimental literature. As demonstrated in their ROC curve analysis (Figure 3), the authors identified that a computationally predicted continuous change of 15% maximized the F1 score for classifying discrete up/down biological regulation. However, the tool currently operates strictly as a retrospective literature-validation engine.



MP-BioPath Cutoff	TP	TN	FP	FP-WD	FP(total)	FN	Accuracy	Sensitivity	Specificity	FPR	Precision	FDR	F1
1%	5604	6544	5556	629	6185	205	0.655303	0.964709933	0.514101658	0.485898	0.475358	0.524642	0.636891
5%	5362	10405	1695	424	2119	652	0.850523	0.891586299	0.830804855	0.169195	0.716749	0.283251	0.794665
10%	5170	11032	1068	370	1438	898	0.872989	0.852010547	0.88468324	0.115317	0.782385	0.217615	0.815725
15%	5005	11294	806	359	1165	1074	0.879221	0.823326205	0.906493298	0.093507	0.811183	0.188817	0.81725
20%	4865	11450	650	290	940	1283	0.880084	0.791314249	0.924132365	0.075868	0.838071	0.161929	0.814022
25%	4702	11504	596	270	866	1466	0.874204	0.76232166	0.929991916	0.070008	0.844468	0.155532	0.801295
30%	4465	11562	538	267	805	1706	0.864548	0.723545617	0.934907415	0.065093	0.847249	0.152751	0.780526
35%	4202	11602	498	255	753	1981	0.852519	0.67960537	0.939053015	0.060947	0.848032	0.151968	0.754534
40%	4069	11617	483	254	737	2115	0.846154	0.657988357	0.940343209	0.059657	0.84665	0.15335	0.740491
45%	3987	11670	430	249	679	2202	0.844589	0.644207465	0.945015791	0.054984	0.854479	0.145521	0.734592
50%	3699	11743	357	242	599	2497	0.832992	0.596998063	0.951466537	0.048533	0.860633	0.139367	0.704974
55%	3356	11753	347	241	588	2841	0.813029	0.541552364	0.952539342	0.047646	0.850913	0.149087	0.661868
60%	3265	11775	325	240	565	2933	0.811307	0.526782833	0.954213938	0.045786	0.85248	0.14752	0.651177
65%	3170	11798	302	196	498	3072	0.807423	0.507850048	0.959499024	0.040501	0.864231	0.135769	0.639758
70%	3095	11801	299	194	493	3149	0.803539	0.495675849	0.959899138	0.040101	0.862598	0.137402	0.629577
75%	3012	11805	295	191	486	3235	0.799277	0.482151433	0.960458872	0.039541	0.861063	0.138937	0.618163
80%	2950	11807	293	189	482	3299	0.796041	0.472075532	0.960777931	0.039222	0.859557	0.140443	0.609441
85%	2907	11810	290	187	477	3344	0.793883	0.465045593	0.961178481	0.038822	0.859043	0.140957	0.603425
90%	2865	11816	284	187	471	3386	0.791941	0.458326668	0.961666802	0.038333	0.858813	0.141187	0.597684
95%	2608	11832	268	185	453	3645	0.778941	0.417079802	0.963125763	0.036874	0.852009	0.147991	0.560017
99%	2135	11842	258	180	438	4123	0.753965	0.341163311	0.964332248	0.035668	0.829771	0.170229	0.483524

Figure 3: The relationship between MP-BioPath sensitivity (TPR) and false positive rate (FPR) at different cutoffs.

2.2 Bottleneck A: The Non-Linear Dynamics of Gene Expression Regulation (GER)

While the 15% cutoff established a strong baseline, the study identified specific biological scenarios where predictive accuracy collapsed. As highlighted in the original author's error analysis (Figure 6), Gene Expression Regulation (GER) discrepancies formed a massive portion of the false negatives.

Unlike rapid Post-Translational Modifications (PTM) which act as binary switches, transcription is inherently quantitative—relying on promoter affinities and mRNA accumulation. Standard logic graphs default to qualitative states, failing to capture the physical reality of transcriptomic accumulation.

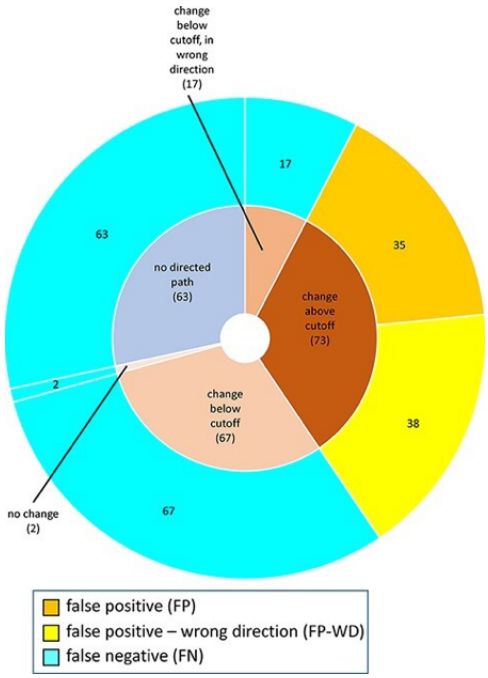


Figure 6: Reasons for discordance between MP-BioPath-based predictions and published evidence, highlighting GER and Entity Set dilution.

2.3 Bottleneck B: Entity Set Dilution & The 1/N Penalty

Figure 6 further highlights "Entity Set" discrepancies. Reactome frequently groups isozymes into unified sets. Currently, MP-BioPath assigns equal mathematical weights to all members. If a target node contains 10 tissue-specific isozymes, but only 1 is physically transcribed in the target tissue, the solver applies an artificial 1/10 weight reduction to the active node. This mathematical dilution dampens the calculated output signal, driving the prediction below the 15% threshold and generating a false negative.

2.4 Bottleneck C: Ubiquitous Substrate Combinatorics (Scale-Free Topologies)

Biological pathways are "scale-free networks," meaning a few nodes act as massive, hyper-connected hubs. From a pure graph-theory perspective, ubiquitous substrates (e.g., ATP, ADP, H2O, Ubiquitin, H^+) create dense algorithmic cycles. When fed into a non-linear optimization solver, these hub nodes cause an exponential increase in the required Jacobian matrix calculations, often leading to solver timeouts. Biologically, however, these substrates are buffered at physiological steady-states (e.g., intracellular ATP at 1-10 mM) and are rarely the rate-limiting perturbation in a specific signal cascade.

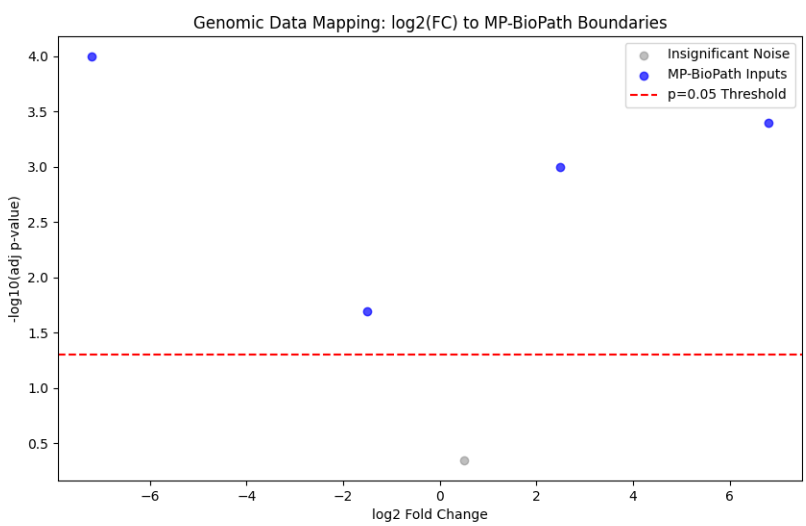
3. Proposed Architecture & Mathematical Models

To solve these bottlenecks, the project will be executed using a polyglot architecture: Python handles high-throughput graph/data manipulation, and Julia (`JuMP.jl`) executes the core non-linear optimization.

3.1 Module A: Genomic Data Ingestion & Boundary Mapping (Python)

MP-BioPath utilizes a mathematical model where node activity values must strictly range from 0.01 (100x decreased activity) to 100.0 (100x increased activity). Standard differential expression pipelines output a continuous $\log_2(\text{Fold Change})$.
Module A translates transcriptomic reality to solver boundaries:

1. **Statistical Filtering:** Eliminate biological noise by dropping genes where $p_{adj} > 0.05$.
2. **Transformation:** Convert $\log_2$ values to absolute multipliers using exponential transformation ($x = 2^{\log_2(\text{FC})}$).
3. **Solver Stability Constraints:** Hard-clip extreme values to exactly $[0.01, 100.0]$ to prevent the JuMP solver from diverging toward infinity during optimization.



Module A: Volcano plot demonstrating statistical filtering (p < 0.05) and boundary mapping of RNA-seq data prior to JuMP solver ingestion.

3.2 Module B: Graph Stitching & API Traversal (Python)

The Stein Lab noted that false negatives spike significantly when biological paths cross arbitrary sub-pathway boundaries, leaving nodes disconnected (Figure 4). Module B will utilize the Reactome ContentService REST API (`/data/pathway/{id}/containedEvents`) to dynamically generate "Super-Logic Graphs."

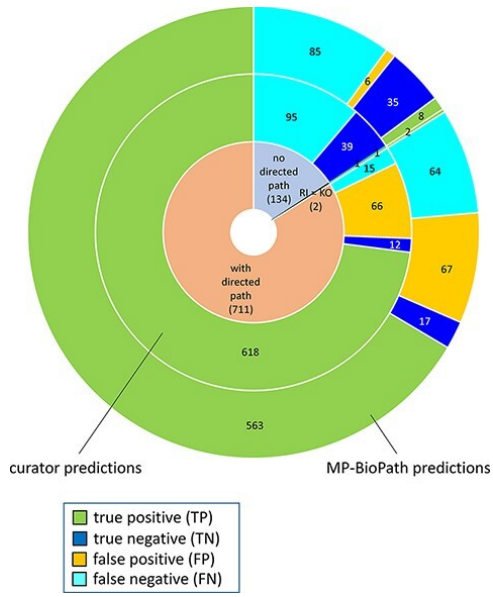


Figure 4: Distribution of predictions with respect to the existence of a directed path, demonstrating the false negative penalty of disconnected sub-pathways.

- **Algorithm:** I will implement a Depth-First Search (DFS) walk-forward algorithm in Python. This will trace flow links across sub-pathway boundaries, dynamically fetching nested JSON logic tables to ensure terminal output nodes maintain mathematical linkage to the perturbed root inputs, explicitly solving the disconnected path issue shown in Figure 4.

3.3 Module C: Expression-Informed Penalty Coefficients (Julia / JuMP.jl)

The core of MP-BioPath minimizes the objective function, heavily penalizing deviations from the ideal calculated state:

$$minimize \sum_{i \in N} w_i(p_i + n_i)$$

Where p_i and n_i are positive/negative deviations, and w_i is the node weight.

The Innovation (Dynamic Weighting): I will update the `JuMP.jl` objective function to incorporate Expression-Informed Penalty Coefficients. By passing the normalized RNA-seq counts from Module A into the Julia environment, the weight w_i of an entity set member will be dynamically scaled:

$$w_i' = w_i \times (\text{Normalized Tissue Expression})$$

This forces the solver to heavily penalize deviations in physically transcribed isozymes while ignoring transcriptionally silent genes, completely resolving the Entity Set Dilution problem mathematically.

3.4 Module D: Matrix Dimensionality Reduction via Heuristic Pruning

To prevent computational bottlenecking, the Python pre-processor will utilize an exclusion dictionary of physiological buffer molecules (ATP, H2O, Pi). Rather than treating these hubs as unknown variables, the algorithm will mathematically constrain them as constants ($x = 1.0$). This domain-specific biological constraint drastically reduces the dimensionality of the matrices processed by the JuMP IPopt solver, improving execution speed and preventing combinatorial timeouts in massive Super-Logic Graphs.

---

4. Proposed 10-Week Timeline (175 Hours)

- **Community Bonding (May):** Finalize Julia/Python polyglot environment; define ubiquitous substrate exclusion lists with mentor; define JSON schemas.
- **Phase 1: Ingestion & API Integration (Weeks 1-4):** Develop `module_a_ingestion.py` for continuous log2 boundaries. Develop `module_b_graphs.py` to interface with the Reactome REST API and apply the Substrate Pruning algorithm (Module D) to the generated JSON graphs.
- **Phase 2: Core Mathematical Upgrades (Weeks 5-7):** Upgrade the `JuMP.jl` objective functions (Module C). Implement dynamic tissue-specific weighting (w_i) to resolve Entity Set Dilution.
- **Phase 3: Validation & Delivery (Weeks 8-10):** Benchmark the integrated pipeline against the "RAF/MAP kinase cascade" (Reactome ID: R-HSA-5673001) to ensure baseline accuracies (~94%) are maintained. Build reporting modules using the 15% biological significance threshold. Generate Dockerfiles and submit final PRs.
    - **Iteration & Stress-Testing:** Perform rigorous validation on the updated algorithm version, ensuring the integration pipeline remains robust across diverse modern genomic datasets.

---

5. About the Candidate

As an undergraduate student in Biochemistry & Molecular Biology, my research focus centers on integrating foundational cellular biology with high-throughput computational models. Through my coursework and independent projects, I have developed a strong quantitative background in RNA-seq differential expression, statistical validation, and dimensionality reduction.
I am comfortable working across multiple programming languages—specifically Python, R, and Julia—which allows me to apply the most effective computational tool to a given bottleneck. Whether the task requires rigorous data pre-processing or complex non-linear mathematical programming, my goal is always to build robust, biologically accurate software solutions.

References

Wright, A.J., Orlic-Milacic, M., Rothfels, K. et al. Evaluating the predictive accuracy of curated biological pathways in a public knowledgebase. *Database* (2022). DOI: 10.1093/database/baac009